A.U.R.A. — Air-Gapped Unified Responsive Agent

Software & System Architecture Documentation

Muhammad Hunzala Hasan (25435) · Muhammad Taha Jabbar (25421) · Hafsa Ayub (25588) | Department of Computer Science, Iqra University | Final Year Project, Batch Fall-2026

1. System Overview

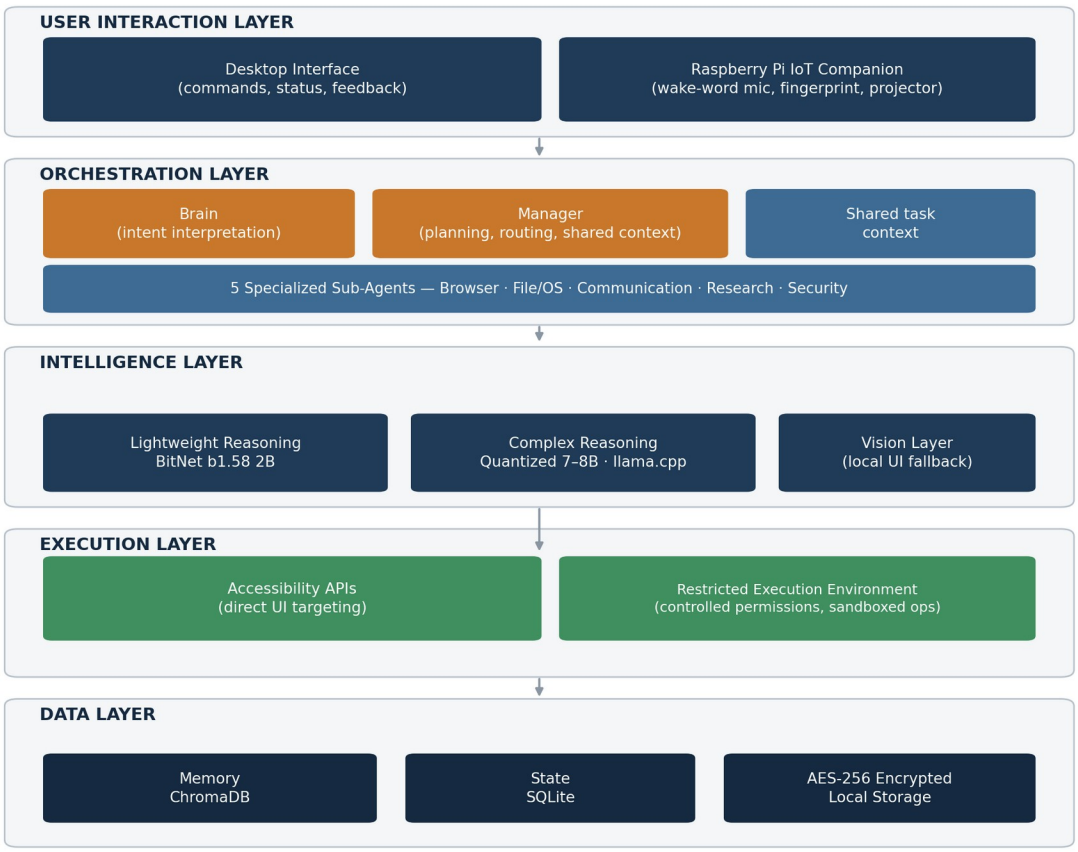
A.U.R.A. is a locally-run, multi-agent software system that converts natural-language commands into controlled operating-system actions — opening files, managing folders, sending messages, and more — without any cloud dependency. All reasoning, memory, and task execution happen on the user's own device; a Raspberry Pi companion node adds hands-free voice input and hardware-enforced biometric approval for high-risk actions. The project spans four domains: Artificial Intelligence (local multi-agent reasoning), Computer Automation (OS/application interaction), Cybersecurity (privacy, access control, agent safety), and Edge Computing (offline, hardware-assisted execution).



Figure 1 — High-level command-to-completion pipeline

2. System Architecture

The system is organized into five layers running on a single device, with no outbound inference calls at any stage. The User Interaction Layer accepts commands via desktop or the IoT companion; the Orchestration Layer (Brain → Manager → five specialized Sub-Agents) plans and routes tasks; the Intelligence Layer provides hybrid local reasoning and vision-based UI fallback; the Execution Layer carries out actions through accessibility APIs inside a restricted sandbox; and the Data Layer persists memory and state under AES-256 encryption.



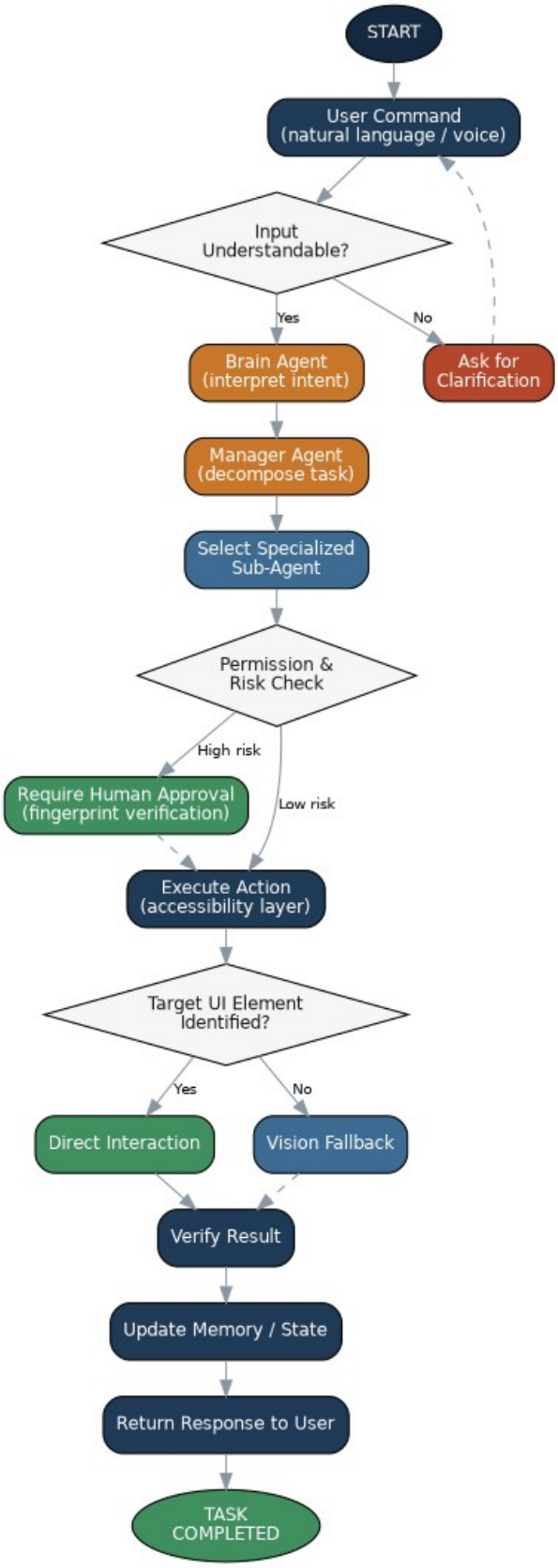
One device · No cloud dependency · No outbound inference calls

3. Core Features

#	Feature	Description
1	**Multi-Agent Orchestration**	Hierarchical Brain → Manager → Sub-Agent structure interprets commands, decomposes tasks, and routes them to five specialized agents (Browser, File/OS, Communication, Research, Security) sharing one task context.
2	**Hybrid Quantized Local Reasoning**	BitNet b1.58 2B handles fast routing/simple decisions; a quantized 7–8B model handles complex reasoning — both fully offline via llama.cpp, no cloud API.
3	**Background OS Automation**	Uses OS accessibility APIs (pywinauto) to target specific UI elements directly, without bringing windows to the foreground or interrupting the user.
4	**Vision Fallback for UI Resilience**	When an element can't be found via the accessibility tree (e.g., after a layout update), a lightweight local vision model relocates it from a screenshot.
5	**Hierarchical Long-Term Memory**	A local vector database (ChromaDB) stores user habits, preferences, and file paths so the system recalls context across sessions.
6	**Zero-Trust Security & Sanitized Sandboxing**	Web content is stripped of HTML/JavaScript before reaching the AI; the agent runs in a restricted-permission sandbox that cannot touch core OS files.
7	**Platform-Compliant Interaction Layer**	Randomized, human-like timing and cursor movement plus per-platform rate limits reduce the risk of automated-behavior detection during controlled testing.
8	**Hardware-in-the-Loop Biometric Security**	High-risk actions (e.g., deleting files, accessing SSH keys) are frozen until approved via a physical fingerprint scan — a hardware, not just software, gate.
9	**Headless IoT Companion Node**	A Raspberry Pi with a wake-word microphone, fingerprint sensor, and projector allows operation and feedback even with the laptop lid closed.
10	**Encrypted Local Data Storage**	All local memory and state databases (ChromaDB + SQLite) are encrypted with AES-256; encryption keys never leave the device.

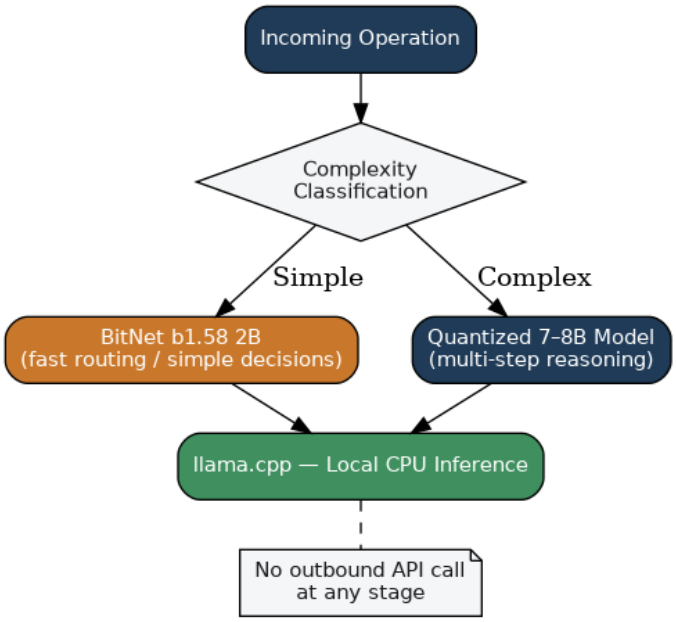
4. End-to-End Process Flow

Every command follows the same controlled path from input to completion. Safety checks — input clarification, permission/risk assessment, and biometric approval for high-risk actions — are built into the flow itself rather than added afterward.


START
User Command
(natural language / voice)
Input
Understandable?
Yes
No
Brain Agent
(interpret intent)
Ask for
Clarification
Manager Agent
(decompose task)
Select Specialized
Sub-Agent
Permission &
Risk Check
High risk
Low risk
Require Human Approval
(fingerprint verification)
Execute Action
(accessibility layer)
Target UI Element
Identified?
Yes
No
Direct Interaction
Vision Fallback
Verify Result
Update Memory / State
Return Response to User
TASK
COMPLETED

5. Hybrid Quantized Reasoning Engine

No native 1.58-bit model above 2B parameters is currently publicly available, so A.U.R.A. uses a two-tier local strategy: BitNet b1.58 2B (ternary {-1, 0, 1} weights, addition-based computation) for fast routing and simple decisions, and a standard 4-bit/8-bit quantized 7–8B model (e.g., Qwen2.5-7B or Llama-3.1-8B) for complex multi-step reasoning. Both run locally via llama.cpp with no outbound API call at any stage — reducing computational overhead, improving CPU-resource use, and making offline operation practical without a dedicated GPU.



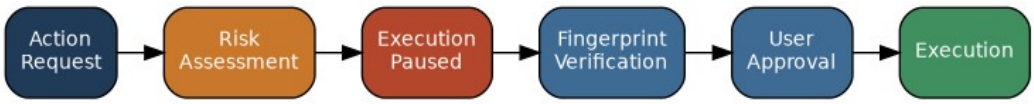
6. Security Architecture

- Local-First Processing — AI inference and user context stay on the local machine; no reliance on cloud AI APIs.
- Zero-Trust Input Handling — external web content is treated as untrusted, reducing the attack surface for indirect prompt injection.
- Restricted Permissions — agents receive no unrestricted OS access; each operation is constrained to what the task needs.
- Human-in-the-Loop — high-risk operations pause and require physical fingerprint approval before executing.
- Encrypted Local Data — memory and state are protected with AES-256; keys never leave the device.
- Auditable Execution — key decisions and actions can be logged so the user can review what was attempted and why.

Untrusted Content Pipeline:



High-Risk Action Pipeline:



7. Technology Stack & Hardware

Category	Components
AI / Reasoning	BitNet b1.58 2B4T; quantized 7–8B model (Qwen2.5-7B / Llama-3.1-8B); served via llama.cpp / Ollama
OS Automation	pywinauto (accessibility APIs), PyAutoGUI, Windows AppContainer
Vision Fallback	Lightweight local vision-language model for screenshot-based UI grounding
Memory & State	ChromaDB (vector database), SQLite (relational state/log store)
Security	AES-256 encryption; zero-trust web-content sanitization; restricted execution sandbox
IoT Hardware	Raspberry Pi 4 Model B / Zero 2W; AS608 optical fingerprint sensor; INMP441 I2S microphone; mini laser/OLED projector
Development	Python; development laptop with 16GB+ RAM, no dedicated GPU required

8. Project Status & Roadmap

Current stage: architecture and design finalized, cross-checked against 50 peer-reviewed / technical sources (2020–2026) spanning multi-agent orchestration, quantized/edge LLM inference, accessibility-based GUI agents, vision-language UI grounding, agent memory systems, and human-in-the-loop agent safety. Implementation has not yet started.

- Phase 1 — Multi-agent architecture (Brain/Manager/Sub-Agents) and hybrid reasoning engine setup.
- Phase 2 — Accessibility-based OS automation module and vision-fallback integration.
- Phase 3 — Long-term memory, zero-trust security layer, and encrypted storage.
- Phase 4 — Raspberry Pi IoT companion node and biometric HITL gate.
- Phase 5 — Integration testing across representative task scenarios and final demo.